Model Evaluation Brief

A repeatable, model-only protocol for comparing three independently developed models

Purpose. Give this brief, one candidate model, and the evaluation dataset to an LLM or evaluator. Repeat the exact same protocol for Member 1, Member 2, and Member 3. The evaluator returns one JSON object per model so the three records can be compared manually.

What to provide for each run

Required input	What it identifies
Candidate model artifact or source, with inference entry point and dependencies	The model being evaluated; include member/model name and version or commit.
One frozen evaluation dataset file, used unchanged in all three runs	Provide the exact file, target column, and fixed test rows/split. The dataset is supplied separately; this PDF does not embed the CSV.
Machine/runtime details	Same hardware, OS, language/runtime, dependency versions, and device (CPU/GPU) for all candidates.

Important: a fair comparison needs the same feature values, column order, label definition, and test examples for all models. Use one agreed, model-ready evaluation file. If a model cannot consume that input as provided, report it as incompatible; do not quietly transform the data to make it run.

Copy this instruction into the LLM

```
Evaluate the attached candidate model using the attached frozen evaluation dataset and the protocol in this PDF. Return exactly one JSON o

Evaluate the model independently. Do not import, call, or depend on any project preprocessing pipeline, preprocessing scripts, pipeline co

Use the same dataset file, target definition, fixed test rows, hardware, runtime, and measurement procedure for every candidate. Calculate
```

Dataset scope

The repository has raw and pre_processed dataset folders, but this brief does not select a dataset for you. Before comparing models, record one exact dataset file and one fixed test subset, then provide that identical file and subset for all three evaluations. The evaluator reads the chosen file; it must not run the repository preprocessing code.

Evaluation rules

1. Keep the experiment identical

- Record dataset filename/version, row count, feature count and order, target column, label mapping, and test-subset identifier. Use the same rows and labels for all three models.
- Do not tune thresholds, choose a different split, rebalance, drop examples, or retrain on the evaluation set. If a model needs different feature engineering, mark the run incompatible instead of giving it a special transformation.
- Record environment and device. Run all candidates on the same machine and under the same conditions when possible. Disclose any unavoidable difference.

2. Report predictive performance

- Always report accuracy, precision, recall, and F1. For multiclass or imbalanced data, report macro and weighted precision, recall, and F1; also include per-class precision, recall, F1, and support. For binary classification, state the positive class and include its precision, recall, and F1.
- Include confusion matrix and class support so the scores can be checked. Use the same label order for all models.
- Report ROC-AUC and PR-AUC only when valid prediction scores/probabilities are available. State binary or multiclass averaging and the score source. Otherwise use null with a short reason.

3. Measure time and complexity

- Measure model load time separately from inference. For inference, use the full fixed test set, 5 warm-up passes, then 10 timed passes; report mean, median (p50), p95 latency per example, and throughput in examples/second. State whether batching is used and its size. Exclude dataset file loading from inference time.
- If stochastic inference is unavoidable, fix and report the seed. Keep the procedure identical across candidates.
- Report training time only when a reproducible training run is part of the supplied artifact and was actually run; otherwise set it to null. Never treat missing training time as zero.
- Report complexity facts available from the artifact: parameter count (or estimable equivalent), model file size, and architecture-specific counts such as layers, tree count/depth, or support vectors. Mark unavailable values null. Do not claim a single complexity score unless a defined measurement is supplied.
- If memory is measured, report peak inference memory and method/device. If it is unavailable, use null.

4. Handle failures and uncertainty honestly

- If execution fails, labels are missing, or shapes/classes do not match, set status to "not_evaluable" or "partial" and state the exact reason. Do not report fabricated metrics.
- Keep raw values at sensible precision (for example, 4-6 decimal places for metrics). Use JSON null for unknown/unavailable values, not strings such as "N/A".
- Return one record for the one candidate only. Do not rank it or name a best model; compare the three JSON records manually after all runs.

Timing note

The fixed 5 warm-ups and 10 timed passes are intended to make inference timing repeatable. Record the actual procedure and do not compare timing numbers gathered under different batching or device conditions.

Required JSON output

Return valid JSON matching this shape. Keep all keys; use null or an empty array/object where a measurement is unavailable. The three model records can then be compared field by field.

```
{
  "schema_version": "1.0",
  "evaluation_status": "complete",
  "model": {
    "member": "Member 1",
    "model_name": "",
    "version_or_commit": "",
    "artifact": "",
    "inference_entry_point": "",
    "embedded_preprocessing": "none or describe"
  },
  "dataset": {
    "filename_or_id": "",
    "version_or_hash": "",
    "rows_evaluated": null,
    "feature_count": null,
    "feature_order_or_schema": "",
    "target_column": "",
    "label_mapping_and_order": [],
    "positive_class": null,
    "fixed_test_split_id": ""
  },
  "environment": {
    "os": "",
    "runtime_and_version": "",
    "framework_and_version": "",
    "device": "CPU/GPU and model",
    "hardware_notes": ""
  },
  "performance": {
    "accuracy": null,
    "precision": {"positive_class": null, "macro": null, "weighted": null},
    "recall": {"positive_class": null, "macro": null, "weighted": null},
    "f1": {"positive_class": null, "macro": null, "weighted": null},
    "roc_auc": {"value": null, "averaging": null, "score_source": null},
    "pr_auc": {"value": null, "averaging": null, "score_source": null},
    "per_class": [{"label": "", "precision": null, "recall": null, "f1": null, "support": null}],
    "confusion_matrix": {"labels_in_order": [], "matrix": []}
  },
  "timing": {
    "model_load_seconds": null,
    "training_seconds": null,
    "inference": {
      "warmup_passes": 5, "timed_passes": 10, "batch_size": null,
      "mean_latency_ms_per_example": null,
      "p50_latency_ms_per_example": null,
      "p95_latency_ms_per_example": null,
      "throughput_examples_per_second": null,
      "measurement_notes": ""
    }
  },
  "complexity": {
    "parameter_count": null,
    "model_file_size_bytes": null,
    "architecture": "",
    "architecture_counts": {},
    "peak_inference_memory_bytes": null,
    "memory_measurement_method": null
  },
  "limitations_or_failure_reason": [],
  "notes": []
}
```

Interpretation

Use the JSON records to compare performance and cost under the same dataset and runtime conditions. Consider F1/recall alongside precision, class-level results, inference time, and complexity. The evaluator records evidence; the project team decides which trade-offs suit the final pipeline.